Operacijski sistemi

Jedro in sistemski klici

Jurij Mihelič, FRI, Uni LJ

Vsebina

- Dostop do jedra OS
- Standardna C knjižnica
- Obravnava napak
- Sledenje izvajanja

Dostop do jedra OS

- **Preko nekaterih ukazov**

`uname`	… izpis informacij od jedru in OS
`arch`	… izpis arhitekture procesorja
`hostname`	… izpis imena gostiteljskega računalnika
`free`	… izpis informacij o pomnilniku
`dmesg`	… izpis sistemskih obvestil
`sysctl -a`	… izpis sistemskih nastavitev

Dostop do jedra OS

- **Preko `/proc`**
  - Preizkusi
    - `man proc`
  - Preglej datoteke
    - `version`, `cmdline`, `uptime`, `modules`
    - `cpuinfo`, `meminfo`
    - `diskstats`, `filesystems`, `mounts`, `partitions`
    - `device`, `dma`, `interrupts`, `iomem`, `ioports`

Dostop do jedra OS

- Demo: `mysysinfo.sh`
  - Napiši skripto za izpis sistemskih informacij
    - npr. ime računalnika, različica OS, št. jedr, velikost pomnilnika, št. procesov itd.
  - Primer izvajanja

```
[jure@lalgec demos]$ ./mysysinfo.sh
Ime računalnika: lalgec.fri.uni-lj.si

Operacijski sistem:
Linux version 3.10.0-1160.6.1.el7.x86_64 (mockbuild@kbuilder.bsys.centos.org) (gcc ver
sion 4.8.5 20150623 (Red Hat 4.8.5-44) (GCC) ) #1 SMP Tue Nov 17 13:59:11 UTC 2020

Št. jedr procesorja: 2

Velikost pomnilnika: 4044424 kB

Št. procesov 151
```

Dostop do jedra OS

- **Preko sistemskih klicev**
  - edini pravi oz. neposredni način
  - izvedba sistemskega klica – s predavanj
    - v zbirniku
    - preko ovojnih funkcij
    - preko funkcije `syscall(…)`
    - posredno preko ostalih funkcij
  - ovojne funkcije sistemskih klicev
    - se nahajajo v standardni C knjižnici

Dostop do jedra OS

- Pregled sistemskih klicev

# Datoteke 1
open()
close()
read()
write()
lseek()
fcntl()
dup()
dup2()

# Datoteke 2
rename()
link()
unlink()
chmod()
chown()
stat()
access()
utime()
truncate()

# Imeniki, ...
chdir()
mkdir()
rmdir()

symlink()
readlink()
mknod()

# Zaščita
getuid()
setuid()
getgid()
setgid()
geteuid()
getegid()
umask()
chroot()

# Procesi
fork()
execve()
wait()
waitpid()
exit()
nice()
times()

# Razno
getpid()
getppid()
time()
sleep()

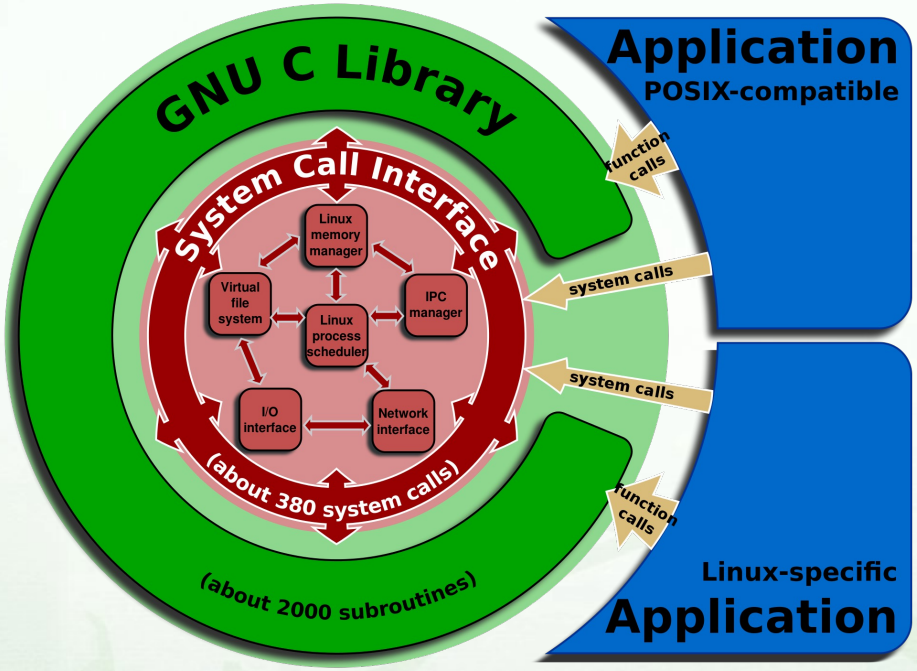
# Naprave
mount()
umount()

# Medprocesna
# komunikacija
pipe()
mmap()
munmap()
shmget()
shmat()

# Signali
kill()
signal()
alarm()
pause()

Standardna C knjižnica

- GNU C knjižnica – glibc
  - standardna knjižnica za jezik C
  - del GNU projekta
  - http://www.gnu.org/s/libc



Standardna C knjižnica

- The GNU C Library Reference Manual
  - https://www.gnu.org/software/libc/manual/html_node/index.html
  - čez 30 poglavij
    - intro, error reporting, memory, characters, strings, locales
    - searching, pattern matching
    - input/output, streams, low-level I/O, files, pipes, sockets
    - terminal, syslog, resource usage
    - mathematics, arithmetic, date&time, crypto
    - programs/system interface (arguments, env. variables)
    - processes, inter-process communication, jobs, threads, signals
    - users, groups

Standardna C knjižnica

- Demo: `environ.c`
  - Napiši program, ki izpiše
    - **ime** samega programa
    - podane **argumente** in
    - **okoljske spremenljivke**

```
int main(int argc, char* argv[], char* environ[])
```

  - Testiraj program
    - Uporabi ukaz `env`

```
gcc -o environ environ.c

env -i a=123 b=456 ./environ 1 2 3
env a=123 b=456 ./environ 1 2 3
```

Standardna C knjižnica

- Demo: `mymkdir-1.c`
  - Napiši program v C, ki naredi naslednje:
    - izpiše pid in ppid procesa
    - izpiše trenutni delovni imenik
      - ne pozabi sprostiti pomnilnika, ki ga morebiti zasede getwd()
    - ustvari imenik z imenom, ki je podan kot 1. argument
      - če imenik ni uspešno ustvarjen, naj se izpiše obvestilo o napaki na standardni izhod za napake
    - skoči v ustvarjeni imenik
    - izpiše trenutni delovni imenik
  - Razmisli
    - Katere funkcije so sistemski klici in katere so višjenivojske?
    - Zakaj po izvedbi programa nismo v ustvarjenem imeniku?

Obravnava napak

- Izhodni status kot rezultat funkcije
  - funkcije običajno vrnejo status izvedbe
    - uspeh: 0
    - neuspeh: -1
  - seveda je nekaj izjem:
    - `getpid()` in `getppid()` vrneta PID
    - `open()` vrne št. datotečnega deskriptorja
    - `read()`, `write()` vrneta št. obdelanih bajtov

Obravnava napak

- Koda zadnje napake
  - spremenljivka `errno`
  - kode napak: `man errno`
- Izpis obvestila o napaki
  - funkcija `perror()`

```
#include <stdlib.h>
#include <errno.h>

int main(int argc, char* argv[]) {
    if (chdir(argv[1]) < 0) {
        int koda = errno;
        perror(argv[0]);
        exit(koda);
    }
    write(1, "Vse je OK.\n", 11);
    exit(0);
}
```

Obravnava napak

- Demo: `mymkdir-2.c`
  - Popravi program `mymkdir-1.c`
    - dodelaj obravnavo napak po vzrocu `errno`
    - uporabi `perror()` za izpis obvestila o napaki
  - Razmisli
    - Kdaj je preverjanje za morebitno napako pri izvedbi nujno, manj nujno, ali pa sploh ni potrebno?

Sledenje izvajanja programa

- Merjenje časa
  - ukaz `time`
- Demo:
  - Poženi nek V/I intenziven proces

```
time ls -R /etc >ls-out.txt
```

- Razmisli
  - Zakaj je čas *real* veliko večji od seštevka *user+sys*?
  - Poženi še enkrat
  - Zakaj je čas *user+sys* sedaj precej manjši?

Sledenje izvajanja programa

- Sledenje sistemskim klicem
  - ukaz `strace`
  - namig:
    - statično prevajanje: `gcc --static`
    - za malce bolj pregleden izpis
- Sledenje klicem v libc
  - ukaz `ltrace`

Sledenje izvajanja programa

- Demo: sledenje `mymkdir-?.c`

```
gcc --static mymkdir-2.c -o mymkdir

strace ./mymkdir

# časovni zaznamki
strace -r ./mymkdir
strace -tt ./mymkdir

# poraba časa
strace -r ./mymkdir

# filter
strace -e open,read,write,close ./mymkdir

# štetje klicev in ostala statistika
strace -c ./ls -R ~ >/dev/null
```